Phase 1: Org foundation, Custom Metadata and Store

Step 1.1: Check your licences

1. Click the **gear icon → Setup**.
2. In **Quick Find**, type Company Information and open it.
3. Note two things:
    - **User Licenses**: the *Salesforce* row's Total, Used and Remaining.
    - **Feature Licenses**: the *Marketing User* row's Total and Remaining.
4. Expect Salesforce = 2 total, 1 used. Tell me if your numbers differ.

Step 1.2: Set locale, time zone and currency (org-wide)

1. Still on **Company Information**, click **Edit**.
2. Set:
    - **Default Locale:** English (India)
    - **Default Language:** English
    - **Default Time Zone:** (GMT+05:30) India Standard Time (Asia/Kolkata)
    - **Currency Locale:** English (India) – INR (if you also see *Default Currency ISO Code*, set it to INR)
3. Click **Save**. If you get a warning, accept it. Your org has no data yet.

Step 1.3: Set your own user's time zone and locale

The nightly reminder job uses the time zone of the user who schedules it, so this matters.

1. Click your **profile picture → Settings**.
2. Go to **My Personal Information → Advanced User Details** and click **Edit**.
3. Set **Time Zone** = (GMT+05:30) India Standard Time (Asia/Kolkata), **Locale** = English (India) and **Language** = English.
4. Click **Save**.

Step 1.4: Set the Indian fiscal year (April–March)

1. In Setup Quick Find, type Fiscal Year and open it.
2. Click **Edit**.
3. Choose **Standard Fiscal Year**, set **Fiscal Year Start Month** = April, and **Fiscal Year is Based On** = *The ending month of the fiscal year*.
4. Click **Save** and accept the warning. This is why a quarter such as Apr–Jun 2026 will report as FY2027 Q1.

Step 1.5: Enable Orders

1. In Quick Find, type Order Settings and open it.
2. Tick **Enable Orders**.
3. Leave every other option unticked, including optional price books, negative quantities, zero quantity and reduction orders. Leaving optional price books off means every order must have a price book, which the design relies on.
4. Click **Save**.

Step 1.6: Deactivate standard duplicate rules

Your own Apex will do the matching, and standard rules can interfere with it.

1. In Quick Find, type Duplicate Rules and open it.
2. For each **active** rule on Lead, Contact or Account, open the rule and click **Deactivate**, then confirm.
3. Leave the matching rules alone.

Step 1.7: Check the State/Country picklist setting (look only)

1. In Quick Find, type State and Country and open **State and Country/Territory Picklists**.
2. Note whether picklists are enabled. **Don't change anything.** My GST formulas will accept "Maharashtra" or "MH" either way. Just remember what you saw for Phase 4.

Step 1.8: Create the Custom Metadata type "Blush Setting"

This holds every business number (GST rates, commission rules and so on) in one place.

1. In Quick Find, type Custom Metadata Types and open it. Click **New Custom Metadata Type**.
2. Enter:
    - **Label:** Blush Setting
    - **Plural Label:** Blush Settings
    - **Object Name:** Blush_Setting (auto-filled)
    - **Visibility:** *Public* (the option saying all Apex code and APIs can use the type and it's visible in Setup)
3. Click **Save**.
4. In the **Custom Fields** section, click **New** and create the fields below. For each Number field, choose **Number**, **Length** = 6, **Decimal Places** = 2, and tick **Required** where noted.

Field Label	Field Name (API)	Type	Notes
Jewellery GST Pct	Jewellery_GST_Pct	Number (6,2)	Required
Garment GST Low Pct	Garment_GST_Low_Pct	Number (6,2)	Required
Garment GST High Pct	Garment_GST_High_Pct	Number (6,2)	Required

Field Label	Field Name (API)	Type	Notes
Garment GST Threshold	Garment_GST_Threshold	Number (6,2)	Required
Wholesale Discount Pct	Wholesale_Discount_Pct	Number (6,2)	Required
Approval Discount Limit Pct	Approval_Discount_Limit_Pct	Number (6,2)	Required
Commission Threshold	Commission_Threshold	Number (6,2)	Required
Commission Pct	Commission_Pct	Number (6,2)	Required
Commission Flat	Commission_Flat	Number (6,2)	Required
Home State	Home_State	Text (50)	Required

Salesforce adds __c to each API name automatically.

5. Click **Manage Blush Settings** (button at the top of the type's page), then **New**.
6. Enter **Label** = Default. **Blush Setting Name** must auto-fill as Default. Don't change it, because formulas and Apex will refer to it by that name.
7. Fill in the values:

Field	Value
Jewellery GST Pct	3
Garment GST Low Pct	5
Garment GST High Pct	12
Garment GST Threshold	1000
Wholesale Discount Pct	30
Approval Discount Limit Pct	10
Commission Threshold	1000
Commission Pct	1
Commission Flat	30
Home State	Maharashtra

8. Click **Save**.

Step 1.9: Create the Store object

1. Setup → **Object Manager** → **Create** → **Custom Object**.
2. Enter:
    - **Label:** Store

- **Plural Label:** Stores
- **Object Name:** Store
- **Record Name → Label:** Store Name, **Data Type:** Text

3. Under **Optional Features**, tick **Allow Reports** and **Allow Search**.
4. Under **Deployment Status**, choose **Deployed**.
5. Leave **Launch New Custom Tab Wizard after saving this custom object** ticked. Click **Save**.
6. In the tab wizard, pick any **Tab Style** (for example a building icon), then click **Next → Next → Save** and accept the defaults.

Step 1.10: Add fields to Store

Go to **Object Manager → Store → Fields & Relationships → New**. For each field, choose the data type, click **Next**, enter the details, then **Next → Next → Save & New**. The default field-level security and page layout choices are fine.

Label	Field Name (API)	Type	Length	Required	Other
Street	Street	Text	255	Yes	
City	City	Text	80	Yes	
State	State	Text	80	Yes	
Postal Code	Postal_Code	Text	10	Yes	
GSTIN	GSTIN	Text	15	Yes	Don't tick *Unique*
Phone	Phone	Phone		Yes	
Website	Website	URL		No	
Active	Active	Checkbox			Default = Checked

Then create the formula field:

1. **New → Formula → Next**.
2. **Field Label:** Full Address, **Field Name:** Full_Address, **Formula Return Type:** *Text*. Click **Next**.
3. Paste this into the formula editor:

```
Street__c & ", " & City__c & ", " & State__c & " - " & Postal_Code__c
```

4. Click **Check Syntax**, then **Next → Next → Save**.

Step 1.11: Add a validation rule to Store

1. **Object Manager → Store → Validation Rules → New**.
2. Enter:
   - **Rule Name:** GSTIN_Must_Be_15_Characters

- **Active:** ticked
- **Error Condition Formula:**

LEN(GSTIN__c) <> 15

- **Error Message:** GSTIN must be exactly 15 characters.
- **Error Location:** Field → GSTIN

3. Click **Save**.

Step 1.12: Create two sample Store records

These use fake GSTINs, so label them as sample data in your documentation.

1. Click the **App Launcher** (nine dots) → search **Stores** → open it.
2. Click **New** and create both records:

Field	Store 1	Store 2
Store Name	Blush - Pune	Blush - Mumbai
Street	Shop 12, FC Road	Unit 5, Linking Road, Bandra West
City	Pune	Mumbai
State	Maharashtra	Maharashtra
Postal Code	411004	400050
GSTIN	27ABCDE1234F1Z5	27ABCDE1234F2Z4
Phone	020-25550101	022-26550202
Website	https://www.blush.example	https://www.blush.example
Active	Checked	Checked

3. Open Store 1 and confirm **Full Address** reads Shop 12, FC Road, Pune, Maharashtra - 411004.

Phase 1 checkpoint

Confirm each of these before moving on:

- Licence numbers noted (Salesforce remaining = 1?)
- Org and your user set to India locale, IST time zone and INR currency
- Fiscal year starts in April, based on the ending month
- Orders enabled
- No active standard duplicate rules on Lead, Contact or Account
- Blush Setting type has 10 fields and a Default record with all values filled
- Store object has 8 fields plus Full_Address and the GSTIN validation rule

- Two Store records exist and the address formula works
- You noted whether State/Country picklists are enabled

Tell me your licence numbers and the State/Country picklist status, plus anything that looked different from my steps. When you're ready, reply **next** for **Phase 2: record types and custom fields**.

Phase 3: Products, price books, data load and the Product2 trigger

By the end of this phase, ticking "Available for Retail/Wholesale" on a product automatically creates (or deactivates) its price book entries. That is your "no admin setup" requirement.

Order of work: fields → price books → Apex class → test class → trigger → load products. The trigger goes in before the data load so that loading the 10 products also tests it.

Pre-check: Dev orgs often come with sample products (such as "GenWatt…"). Open **App Launcher → Products** and look at **All Products**. If any exist, untick **Active** on each one, or delete them. This keeps them away from our price books.

Step 3.1: Create the Product fields

Go to **Setup → Object Manager → Product → Fields & Relationships → New**. At the page-layout step, leave **Product Layout** ticked.

Label	Field Name	Type	Details
Category	Category	Picklist	Values: Garments, Jewellery. Tick **Required**. Don't tick "Use first value as default". Tick **Restrict picklist to the values defined in the value set**
Cost Price	Cost_Price	Currency	Length 16, Decimal Places 2. Tick **Required**
Retail Price	Retail_Price	Currency	Length 16, Decimal Places 2. Tick **Required**
Available for Retail	Available_for_Retail	Checkbox	Default Unchecked
Available for Wholesale	Available_for_Wholesale	Checkbox	Default Unchecked

Then create the formula field:

1. **New → Formula → Next**.
2. **Field Label:** Wholesale Price, **Field Name:** Wholesale_Price, **Return Type:** *Currency*, **Decimal Places:** 2. Click **Next**.
3. Formula:

```
ROUND(Retail_Price__c * (100 -
$CustomMetadata.Blush_Setting__mdt.Default.Wholesale_Discount_Pct__c) / 100, 2)
```

4. **Check Syntax → Next → Next → Save**.

If the syntax check fails on the $CustomMetadata part, use the **Insert Field** button, expand **$CustomMetadata → Blush Setting → Default** and pick **Wholesale Discount Pct**. Then tell me what the editor inserted.

Step 3.2: Product validation rules, layout and list views

Validation rules

Object Manager → Product → Validation Rules → New. Create three rules.

Rule Name	Error Condition Formula	Error Message	Error Location
Retail_Price_Must_Be_Positive	Retail_Price__c <= 0	Retail price must be greater than zero.	Field → Retail Price
Cost_Price_Not_Negative	Cost_Price__c < 0	Cost price cannot be negative.	Field → Cost Price
Product_Code_Required	ISBLANK(ProductCode)	Product Code is required (it appears on invoices).	Field → Product Code

Tick **Active** on each.

Page layout

1. **Object Manager → Product → Page Layouts → Product Layout** (click the name).
2. Make sure the new fields (Category, Cost Price, Retail Price, Wholesale Price, Available for Retail, Available for Wholesale) sit in the **Product Information** section. Drag them from the palette if any are missing.
3. **Save**.

List views

App Launcher → Products → list view dropdown → New. For each, set **All users** and click **Save**.

List Name	Filter
Retail Products	Available for Retail equals True
Wholesale Products	Available for Wholesale equals True

Add the columns Product Name, Product Code, Category, Retail Price, Wholesale Price, Active through the **gear icon → Select Fields to Display**.

Step 3.3: Create the price books

1. **App Launcher** → search **Price Books** → open it.
2. Click **Standard Price Book**. Confirm **Active** is ticked. If it isn't, click **Edit**, tick it and **Save**.

3. Go back to the list and click **New**:
    - **Price Book Name:** Retail Price Book (exact spelling and spacing, because the code looks it up by name)
    - **Active:** ticked
    - **Description:** Retail selling prices
    - **Save**
4. **New** again: **Price Book Name:** Wholesale Price Book, **Active** ticked, **Description:** Wholesale selling prices (30% below retail). **Save**.

Don't add any products to them by hand.

Step 3.4: Apex handler class

1. Click the **gear icon → Developer Console**. A new window opens.
2. **File → New → Apex Class**. Name it ProductPricebookSync and click **OK**.
3. Delete the template and paste this whole class:

apex

```
public with sharing class ProductPricebookSync {


  public static final String RETAIL_BOOK = 'Retail Price Book';
  public static final String WHOLESALE_BOOK = 'Wholesale Price Book';


  // Called by ProductTrigger after insert and after update.
  public static void handle(List<Product2> newList, Map<Id, Product2> oldMap) {


    // 1. Keep only products where something relevant is new or changed
    List<Product2> changed = new List<Product2>();
    for (Product2 p : newList) {
      Product2 old = (oldMap == null) ? null : oldMap.get(p.Id);
      if (old == null
        || p.Retail_Price__c != old.Retail_Price__c
        || p.Available_for_Retail__c != old.Available_for_Retail__c
        || p.Available_for_Wholesale__c != old.Available_for_Wholesale__c
        || p.IsActive != old.IsActive) {
```

```
            changed.add(p);
        }
    }
    if (changed.isEmpty()) {
        return;
    }

    // 2. Find the two custom price books by name
    List<String> bookNames = new List<String>{ RETAIL_BOOK, WHOLESALE_BOOK };
    Map<String, Id> bookIds = new Map<String, Id>();
    for (Pricebook2 pb : [SELECT Id, Name FROM Pricebook2 WHERE Name IN :bookNames]) {
        bookIds.put(pb.Name, pb.Id);
    }
    if (!bookIds.containsKey(RETAIL_BOOK) || !bookIds.containsKey(WHOLESALE_BOOK)) {
        for (Product2 p : changed) {
            p.addError('Price books "' + RETAIL_BOOK + '" and "' + WHOLESALE_BOOK
                + '" must exist before products can be saved.');
        }
        return;
    }
    Id retailId = bookIds.get(RETAIL_BOOK);
    Id wholesaleId = bookIds.get(WHOLESALE_BOOK);
    // Tests cannot query the standard price book, so they use the Test method.
    Id standardId = Test.isRunningTest()
        ? Test.getStandardPricebookId()
        : [SELECT Id FROM Pricebook2 WHERE IsStandard = true LIMIT 1].Id;

    // 3. Wholesale discount % comes from the Blush Setting record (falls back to 30)
    Blush_Setting__mdt setting = Blush_Setting__mdt.getInstance('Default');
    Decimal discountPct = (setting == null) ? 30 : setting.Wholesale_Discount_Pct__c;
```

```
    // 4. Load the entries that already exist for these products
    Set<Id> productIds = new Map<Id, Product2>(changed).keySet();
    Set<Id> allBooks = new Set<Id>{ standardId, retailId, wholesaleId };
    Map<String, PricebookEntry> existing = new Map<String, PricebookEntry>();
    for (PricebookEntry e : [
      SELECT Id, Product2Id, Pricebook2Id, UnitPrice, IsActive
      FROM PricebookEntry
      WHERE Product2Id IN :productIds AND Pricebook2Id IN :allBooks
    ]) {
      existing.put(key(e.Product2Id, e.Pricebook2Id), e);
    }

    // 5. Decide what to insert and what to update
    List<PricebookEntry> standardInserts = new List<PricebookEntry>();
    List<PricebookEntry> otherInserts = new List<PricebookEntry>();
    Map<Id, PricebookEntry> updates = new Map<Id, PricebookEntry>();

    for (Product2 p : changed) {
      Decimal retail = p.Retail_Price__c.setScale(2, RoundingMode.HALF_UP);
      Decimal wholesale = retail.multiply(100 - discountPct).divide(100, 2,
RoundingMode.HALF_UP);

      plan(p.Id, standardId, retail, p.IsActive, existing, standardInserts, updates);
      plan(p.Id, retailId, retail, p.IsActive && p.Available_for_Retail__c,
        existing, otherInserts, updates);
      plan(p.Id, wholesaleId, wholesale, p.IsActive && p.Available_for_Wholesale__c,
        existing, otherInserts, updates);
    }

    // 6. Standard entries must exist before entries in other price books
    insert standardInserts;
```

```
        insert otherInserts;
        update updates.values();
    }

    private static void plan(Id productId, Id bookId, Decimal price, Boolean shouldBeActive,
                             Map<String, PricebookEntry> existing,
                             List<PricebookEntry> inserts,
                             Map<Id, PricebookEntry> updates) {
        PricebookEntry pbe = existing.get(key(productId, bookId));
        if (pbe == null) {
            if (shouldBeActive) {
                inserts.add(new PricebookEntry(
                    Pricebook2Id = bookId, Product2Id = productId,
                    UnitPrice = price, IsActive = true));
            }
            return;
        }
        // Entries are never deleted (old orders may use them); they are deactivated instead.
        if (pbe.UnitPrice != price || pbe.IsActive != shouldBeActive) {
            pbe.UnitPrice = price;
            pbe.IsActive = shouldBeActive;
            updates.put(pbe.Id, pbe);
        }
    }

    private static String key(Id productId, Id bookId) {
        return productId + '|' + bookId;
    }
}
```

4. Press **Ctrl+S** to save and compile. Open the **Problems** tab at the bottom: it should be empty.

If you see "Variable does not exist: Retail_Price__c" (or similar): a field API name differs from Step 3.1. Fix the field name in Object Manager. Don't edit the code.

Step 3.5: Apex test class

1. **File → New → Apex Class**, name ProductPricebookSyncTest.
2. Replace the template with:

apex

```
@IsTest
private class ProductPricebookSyncTest {

    @TestSetup
    static void setup() {
        insert new List<Pricebook2>{
            new Pricebook2(Name = ProductPricebookSync.RETAIL_BOOK, IsActive = true),
            new Pricebook2(Name = ProductPricebookSync.WHOLESALE_BOOK, IsActive = true)
        };
    }

    private static Product2 buildProduct(String code, Decimal retail, Boolean forRetail, Boolean forWholesale) {
        return new Product2(
            Name = 'Test ' + code, ProductCode = code, Category__c = 'Garments',
            Cost_Price__c = retail / 2, Retail_Price__c = retail,
            Available_for_Retail__c = forRetail, Available_for_Wholesale__c = forWholesale,
            IsActive = true);
    }

    // Expected wholesale price using the same setting the trigger reads
    private static Decimal wholesaleOf(Decimal retail) {
        Decimal pct = Blush_Setting__mdt.getInstance('Default').Wholesale_Discount_Pct__c;
        return retail.multiply(100 - pct).divide(100, 2, RoundingMode.HALF_UP);
    }
```

```
// Returns entries keyed 'Standard', 'Retail' or 'Wholesale'
private static Map<String, PricebookEntry> entriesFor(Id productId) {
    Map<String, PricebookEntry> result = new Map<String, PricebookEntry>();
    Id stdId = Test.getStandardPricebookId();
    for (PricebookEntry e : [
        SELECT Pricebook2Id, Pricebook2.Name, UnitPrice, IsActive
        FROM PricebookEntry WHERE Product2Id = :productId
    ]) {
        String label = 'Wholesale';
        if (e.Pricebook2Id == stdId) {
            label = 'Standard';
        } else if (e.Pricebook2.Name == ProductPricebookSync.RETAIL_BOOK) {
            label = 'Retail';
        }
        result.put(label, e);
    }
    return result;
}

@IsTest
static void insertSellableInBothChannels() {
    Product2 p = buildProduct('T#001', 1000, true, true);
    insert p;
    Map<String, PricebookEntry> e = entriesFor(p.Id);
    Assert.areEqual(3, e.size(), 'Standard, Retail and Wholesale entries expected');
    Assert.areEqual(1000.00, e.get('Standard').UnitPrice);
    Assert.areEqual(1000.00, e.get('Retail').UnitPrice);
    Assert.areEqual(wholesaleOf(1000), e.get('Wholesale').UnitPrice);
    Assert.isTrue(e.get('Retail').IsActive);
    Assert.isTrue(e.get('Wholesale').IsActive);
```

```
}

@IsTest
static void insertRetailOnlyCreatesNoWholesaleEntry() {
    Product2 p = buildProduct('T#002', 500, true, false);
    insert p;
    Map<String, PricebookEntry> e = entriesFor(p.Id);
    Assert.areEqual(2, e.size());
    Assert.isTrue(e.containsKey('Standard'));
    Assert.isTrue(e.containsKey('Retail'));
    Assert.isFalse(e.containsKey('Wholesale'));
}

@IsTest
static void untickingChannelDeactivatesEntry() {
    Product2 p = buildProduct('T#003', 1000, true, true);
    insert p;
    p.Available_for_Wholesale__c = false;
    update p;
    Map<String, PricebookEntry> e = entriesFor(p.Id);
    Assert.areEqual(3, e.size(), 'Entry must be kept, not deleted');
    Assert.isFalse(e.get('Wholesale').IsActive);
    Assert.isTrue(e.get('Retail').IsActive);
}

@IsTest
static void tickingChannelLaterCreatesEntry() {
    Product2 p = buildProduct('T#004', 1000, true, false);
    insert p;
    p.Available_for_Wholesale__c = true;
    update p;
```

```
    Map<String, PricebookEntry> e = entriesFor(p.Id);
    Assert.areEqual(3, e.size());
    Assert.isTrue(e.get('Wholesale').IsActive);
    Assert.areEqual(wholesaleOf(1000), e.get('Wholesale').UnitPrice);
}

@IsTest
static void tickingChannelAgainReactivatesEntry() {
    Product2 p = buildProduct('T#005', 1000, true, true);
    insert p;
    p.Available_for_Wholesale__c = false;
    update p;
    p.Available_for_Wholesale__c = true;
    update p;
    Map<String, PricebookEntry> e = entriesFor(p.Id);
    Assert.areEqual(3, e.size(), 'No duplicate entry may be created');
    Assert.isTrue(e.get('Wholesale').IsActive);
}

@IsTest
static void priceChangeUpdatesAllEntries() {
    Product2 p = buildProduct('T#006', 1000, true, true);
    insert p;
    p.Retail_Price__c = 1200;
    update p;
    Map<String, PricebookEntry> e = entriesFor(p.Id);
    Assert.areEqual(1200.00, e.get('Standard').UnitPrice);
    Assert.areEqual(1200.00, e.get('Retail').UnitPrice);
    Assert.areEqual(wholesaleOf(1200), e.get('Wholesale').UnitPrice);
}
```

```
@IsTest
static void deactivatingProductDeactivatesEntries() {
    Product2 p = buildProduct('T#007', 1000, true, true);
    insert p;
    p.IsActive = false;
    update p;
    for (PricebookEntry e : entriesFor(p.Id).values()) {
        Assert.isFalse(e.IsActive, 'All entries must be inactive');
    }
}

@IsTest
static void wholesaleRoundsHalfUp() {
    // 100.05 x 70% = 70.035 -> 70.04 (assumes the Default setting is 30%)
    Product2 p = buildProduct('T#008', 100.05, true, true);
    insert p;
    Assert.areEqual(70.04, entriesFor(p.Id).get('Wholesale').UnitPrice);
}

@IsTest
static void unrelatedChangeDoesNothing() {
    Product2 p = buildProduct('T#009', 1000, true, true);
    insert p;
    p.Name = 'Renamed product';
    update p;
    Assert.areEqual(3, entriesFor(p.Id).size());
}

@IsTest
static void bulkInsertOf200Products() {
    List<Product2> products = new List<Product2>();
```

```
        for (Integer i = 0; i < 200; i++) {
            products.add(buildProduct('B#' + i, 100 + i, true, true));
        }
        insert products;
        Integer entryCount = [SELECT COUNT() FROM PricebookEntry WHERE Product2Id IN :products];
        Assert.areEqual(600, entryCount);
    }

    @IsTest
    static void missingPriceBookBlocksSave() {
        delete [SELECT Id FROM Pricebook2 WHERE Name = :ProductPricebookSync.RETAIL_BOOK];
        Database.SaveResult sr = Database.insert(buildProduct('T#010', 1000, true, true), false);
        Assert.isFalse(sr.isSuccess());
        Assert.isTrue(sr.getErrors()[0].getMessage().contains('Retail Price Book'));
    }
}
```

3. **Ctrl+S**. Check the **Problems** tab is empty.

Run the tests

1. **Test → New Run**.
2. Under **Test Classes**, select ProductPricebookSyncTest, click **Add Selected**, then **Run**.
3. The tests will fail until the trigger exists. Do Step 3.6 first, then run them.

Step 3.6: The trigger

1. **File → New → Apex Trigger**.
2. **Name:** ProductTrigger, **sObject:** Product2. Click **Submit**.
3. Replace the template with:

```
apex
trigger ProductTrigger on Product2 (after insert, after update) {
    ProductPricebookSync.handle(Trigger.new, Trigger.oldMap);
}
```

4. **Ctrl+S**.

Now run the tests

1. **Test → New Run → ProductPricebookSyncTest → Add Selected → Run**.
2. Wait for the results. Expected: **11 passed, 0 failed**.
3. Check coverage: tick **Show Code Coverage** (lower area), or open the **Tests** tab → **Overall Code Coverage**. ProductPricebookSync should show **100%**.

If a test fails, double-click it to see the message. Typical causes:

- a field API name mismatch (fix the field, not the code)
- the Default metadata record missing or unnamed
- wholesaleRoundsHalfUp failing because the setting isn't exactly 30

Send me the failure text if you can't resolve it.

Step 3.7: Load the 10 products

Option A: Data Loader (recommended)

1. Create a file products.csv (Notepad or Excel, saved as **CSV**) with exactly this content:

Name,ProductCode,Category__c,Cost_Price__c,Retail_Price__c,Available_for_Retail__c,Available_for_Wholesale__c,IsActive

Jeans,10#025,Garments,750,1500,TRUE,TRUE,TRUE

Top,10#026,Garments,500,1000,TRUE,TRUE,TRUE

Shorts,10#030,Garments,275,550,TRUE,TRUE,TRUE

Jacket,11#121,Garments,450,900,TRUE,TRUE,TRUE

Tshirt,11#122,Garments,250,500,TRUE,TRUE,TRUE

Gown,12#013,Garments,600,1200,TRUE,TRUE,TRUE

Earrings,13#014,Jewellery,200,400,TRUE,TRUE,TRUE

Earrings,14#004,Jewellery,100,200,TRUE,TRUE,TRUE

Necklace Set,14#005,Jewellery,1250,2500,TRUE,TRUE,TRUE

Necklace,14#006,Jewellery,700,1400,TRUE,TRUE,TRUE

These are the 10 valid rows from your sample. The blank 11th row is left out, and "Jwellery" is corrected to "Jewellery".
2. In Setup, search **Data Loader** and download the installer for your computer. Install it.
3. Open Data Loader → **Insert**.
4. Log in using **OAuth** (environment: **Production**) with your Developer Org username and password, then **Allow**.
5. Tick **Show all Salesforce objects**, select **Product (Product2)**, **Browse** to products.csv, click **Next**, then **OK** (it should say 10 records).

6. **Create or Edit a Map → Auto-Match Fields to Columns → OK → Next**.
7. Choose any output folder → **Finish**. It should report **10 successes, 0 errors**.

Option B: By hand

App Launcher → Products → New, once per row, using the values from the CSV above. Tick **Active** each time. Ten records is quick, and each save fires the trigger.

Step 3.8: Verify the load

1. **Products → All Products**: 10 products (plus any sample ones you made inactive).
2. Open **Jeans → Related** tab → **Price Books**. It should show 3 entries. Check the other products against this table:

Product (Code)	Standard	Retail	Wholesale
Jeans (10#025)	1,500	1,500	1,050
Top (10#026)	1,000	1,000	700
Shorts (10#030)	550	550	385
Jacket (11#121)	900	900	630
Tshirt (11#122)	500	500	350
Gown (12#013)	1,200	1,200	840
Earrings (13#014)	400	400	280
Earrings (14#004)	200	200	140
Necklace Set (14#005)	2,500	2,500	1,750
Necklace (14#006)	1,400	1,400	980

3. Open **Price Books → Retail Price Book → Related → Products**: 10 entries. Do the same for **Wholesale Price Book**.
4. Open any product and check that the **Wholesale Price** formula field matches the table.

Step 3.9: Add the retail-only demo product and test channel switching

1. **Products → New**:
    - **Product Name:** Scarf
    - **Product Code:** 99#001
    - **Category:** Garments
    - **Cost Price:** 150

    - **Retail Price:** 300
    - **Active:** ticked
    - **Available for Retail:** ticked
    - **Available for Wholesale:** unticked
2. **Save**, then open **Related → Price Books**. Expected: Standard 300 and Retail 300 only.
3. Edit Scarf, tick **Available for Wholesale**, **Save**. Now there should be a Wholesale entry of **210**.
4. Edit again and untick **Available for Wholesale**, **Save**. The Wholesale entry must still be listed, but shown as **inactive**.
5. Edit and change **Retail Price** to 400. Standard and Retail become 400, and Wholesale becomes 280.
6. Set the price back to 300 and leave Scarf as **retail-only**.

Phase 3 checkpoint

- Product has 5 custom fields and the Wholesale_Price formula
- 3 validation rules active; layout and 2 list views done
- Retail Price Book and Wholesale Price Book exist and are active; Standard is active
- Classes ProductPricebookSync and ProductPricebookSyncTest, and trigger ProductTrigger, saved without errors
- 11 tests pass and coverage on ProductPricebookSync is 100%
- 10 sample products loaded, each with Standard, Retail and Wholesale entries matching the table
- Scarf (99#001) behaves correctly when you tick, untick and change the price

Tell me your test results (passed count and coverage %) and anything that differed, especially from Steps 3.1 and 3.6. Then reply **next** for **Phase 4: the Order data model** (record types, statuses, fields, roll-ups and the Order Product fields).

Next

Phase 4: The Order data model

You'll configure Order statuses and record types, then add the fields, roll-ups and layouts that Phase 5's automation will fill in. Nothing calculates yet, so the line-level values stay at zero until Phase 5.

Order of work matters: statuses → record types → Order Product fields → Order fields (roll-ups need the Order Product fields to exist first) → layouts → test.

Step 4.1: Order Status values

The design maps **Paid** to the “Activated” category, which Salesforce locks against edits. Everything else is Draft category.

1. **Setup → Object Manager → Order → Fields & Relationships → Status** (click the field name).
2. Find the **Status Picklist Values** section. You should see **Draft** and **Activated**, each with a status category.
3. Click **New** and add these two values, one at a time. Set **Status Category = Draft** for both:
    - Confirmed
    - Cancelled
4. Click **Edit** beside **Activated**. Change the **Label** to Paid. If an **API Name** box is shown, set it to Paid as well. Leave the category as **Activated**. Click **Save**.
5. Go back to the Status field and check the final list:

Value	Status Category
Draft	Draft
Confirmed	Draft
Cancelled	Draft
Paid	Activated

Tell me what you see for the Paid value’s API name. Later formulas use ISPICKVAL(Status, "Paid"). If the API name stayed “Activated”, I’ll switch those formulas.

Step 4.2: Order record types

Clone two layouts

Object Manager → Order → Page Layouts. Clone **Order Layout** twice using the ▼ menu: Retail Order Layout and Wholesale Order Layout.

Create the record types

Order → Record Types → New, twice.

Setting	Retail	Wholesale
Existing Record Type	Master	Master
Label / Name	Retail / Retail	Wholesale / Wholesale
Active	ticked	ticked
System Administrator	Make Available + Make Default	Make Available
Layout	Retail Order Layout	Wholesale Order Layout

The new record types should include all four Status values automatically. After saving each, open it and check that the **Status** picklist lists Draft, Confirmed, Cancelled and Paid. If any is missing, click **Edit** next to Status and add it.

Step 4.3: Confirm roll-ups are possible (check only)

1. **Object Manager → Order → Fields & Relationships → New**.
2. Look at the field type list. **Roll-Up Summary** should be offered.
3. Click **Cancel**.

If Roll-Up Summary is missing, stop and tell me. The design would need to change.

Step 4.4: Order Product fields

Object Manager → Order Product → Fields & Relationships → New. At the layout step, keep **Order Product Layout** ticked.

Label	Field Name	Type	Details
Discount %	Discount_Percent	Number	Length 3, Decimal Places 2. **Required**, Default Value 0. Help text: Enter 10 for 10%
Discount Amount	Discount_Amount	Currency	16, 2. Help text: Per unit
Taxable Amount	Taxable_Amount	Currency	16, 2
GST Rate %	GST_Rate	Number	Length 3, Decimal Places 2
GST Amount	GST_Amount	Currency	16, 2
CGST Amount	CGST_Amount	Currency	16, 2
SGST Amount	SGST_Amount	Currency	16, 2
IGST Amount	IGST_Amount	Currency	16, 2
Net Price	Net_Price	Currency	16, 2
Cost Amount	Cost_Amount	Currency	16, 2

These are all plain stored fields, not formulas, which is what allows them to be rolled up. Phase 5's Flow will fill them.

Step 4.5: Order custom fields

Object Manager → Order → Fields & Relationships → New. Leave the default layout selections (all Order layouts ticked).

4.5.1 Standard types

Label	Field Name	Type	Details
Store	Store	Lookup → Store	Child Relationship Name: Orders. Related list label: Orders
Sales Representative	Sales_Representative	Lookup → User	Child Relationship Name: Orders_Handled. Related list label: Orders Handled
Source Opportunity	Source_Opportunity	Lookup → Opportunity	Child Relationship Name: Orders_From_Opportunity. Related list label: Blush Orders
Payment Mode	Payment_Mode	Picklist	Values: Cash, Card, UPI, Bank Transfer. Don't make it required (a validation rule will handle that)
Invoice Number	Invoice_Number	Auto Number	Display Format INV-{0000}, Starting Number 1
Approved Max Discount %	Approved_Max_Discount	Number	Length 3, Decimals 2, Default 0
Commission Amount	Commission_Amount	Currency	16, 2

If the lookup wizard shows an extra "lookup filter" screen, skip it.

4.5.2 Roll-up summary fields

Choose **Roll-Up Summary → Next**. For each, set the **Summarized Object** to **Order Products** and leave the filter as **All records should be included**.

Label	Field Name	Roll-up Type	Field to Aggregate
Line Count	Line_Count	COUNT	(none)
Max Discount %	Max_Discount	MAX	Discount %
Effective Amount	Effective_Amount	SUM	Taxable Amount
Total CGST	Total_CGST	SUM	CGST Amount
Total SGST	Total_SGST	SUM	SGST Amount
Total IGST	Total_IGST	SUM	IGST Amount
Total GST	Total_GST	SUM	GST Amount

Label	Field Name	Roll-up Type	Field to Aggregate
Total Payable	Total_Payable	SUM	Net Price
Total Cost	Total_Cost	SUM	Cost Amount

Set **Decimal Places** to 2 on the currency-type roll-ups and 0 on **Line Count**, where the wizard asks.

4.5.3 Formula fields

Create these after the roll-ups, because they reference them. Each: **New → Formula → Next**, pick the return type, **Next**, paste the formula, **Check Syntax → Next → Next → Save & New**.

Intra State (Intra_State, return type **Checkbox**)

```
OR(
  UPPER(TRIM(BillingState)) =
UPPER($CustomMetadata.Blush_Setting__mdt.Default.Home_State__c),
  UPPER(TRIM(BillingState)) = "MH"
)
```

Commission Calculated (Commission_Calculated, **Currency**, 2 decimals)

```
IF(Line_Count__c = 0, 0,
  IF(Effective_Amount__c <=
$CustomMetadata.Blush_Setting__mdt.Default.Commission_Threshold__c,
    ROUND(Effective_Amount__c *
$CustomMetadata.Blush_Setting__mdt.Default.Commission_Pct__c / 100, 2),
    $CustomMetadata.Blush_Setting__mdt.Default.Commission_Flat__c))
```

Gross Profit (Gross_Profit, **Currency**, 2 decimals)

```
Effective_Amount__c - Total_Cost__c
```

If **Intra State** fails the syntax check on BillingState, tell me and send the exact error.

4.5.4 Relabel the date field

1. **Order → Fields & Relationships → Order Start Date → Edit**.
2. Change the **Field Label** to Order Date. **Save**.

This is only a label change. The API name stays the same, and all reports and invoices will use “Order Date”.

Step 4.6: Order Product page layout

1. **Object Manager → Order Product → Page Layouts → Order Product Layout**.
2. Drag all 10 new fields into the field section from the palette.

3. For each calculated field (everything except **Discount %**), click the **wrench icon** on the field and tick **Read-Only**. Click **OK**.
4. **Save**.

Step 4.7: Order page layouts

Do this for **both** Retail Order Layout and Wholesale Order Layout.

1. Open the layout. Click the **Fields** palette item if it isn't showing.
2. Click **Create new section** (or drag the "Section" item from the palette). Name it Blush Order Details, 2 columns. Add: **Store, Sales Representative, Payment Mode, Invoice Number, Source Opportunity**.
3. Add a second section called Totals and Commission with: **Max Discount %, Approved Max Discount %, Effective Amount, Total CGST, Total SGST, Total IGST, Total GST, Total Payable, Total Cost, Gross Profit, Commission Calculated, Commission Amount, Intra State**.
4. Set **Read-Only** (wrench icon) on **Invoice Number, Line Count (if added), Max Discount %, Approved Max Discount %, Effective Amount, Total CGST/SGST/IGST/GST, Total Payable, Total Cost, Gross Profit, Commission Calculated, Commission Amount, Intra State**.
5. Check that these standard fields are on the layout: **Account Name, Order Date, Status, Price Book, Billing Address, Record Type**. Drag any that are missing.
6. In **Related Lists**, drag **Approval History** onto the layout. Phase 5 uses it.
7. Click the **wrench** on the **Order Products** related list and select these columns, in this order: **Product Name, Product Code, Quantity, List Price, Unit Price, Discount %, Discount Amount, Taxable Amount, GST Rate %, GST Amount, Net Price**. Click **OK**.
8. **Save**.

Step 4.8: Compact layout, list views and Path

Compact layout

Object Manager → Order → Compact Layouts → New.

- **Label:** Blush Order Compact
- **Fields in order:** Order Number, Account Name, Record Type, Status, Total Payable, Invoice Number
- **Save**, then **Compact Layout Assignment → Edit Assignment → Primary = Blush Order Compact → Save**.

List views

App Launcher → Orders → list view dropdown → New. For each, set **All users** and filter owner by **All orders**.

List Name	Additional filter
Retail Orders	Record Type equals Retail
Wholesale Orders	Record Type equals Wholesale
Awaiting Payment	Status equals Confirmed

Status Path

This gives Sales a clickable status bar, which is how they will mark an order Paid.

1. **Setup → Path Settings → New Path.**
2. **Path Name:** Retail Order Path. **API Name:** Retail_Order_Path. **Object:** Order. **Record Type:** Retail. **Picklist:** Status. **Next**.
3. Tick **Draft, Confirmed, Paid** as the stages in this order. Cancelled sits outside the main flow, so leave it off the path.
4. **Next → Next → Activate**. Click **Finish**.
5. Repeat for Wholesale Order Path with **Record Type: Wholesale**.

The path bar will also appear on the Order record page once the Blush app's pages are set up in Phase 10. If it doesn't appear on the Order page by default, we'll add it there.

Step 4.9: Test the Order model

Test A: Manual order with a product

1. **App Launcher → Orders → New**, record type **Retail**.
2. Enter **Account:** Test Retail Customer (from Phase 2), **Order Date:** today, **Status:** Draft, **Price Book:** Retail Price Book.
3. Fill **Billing State** with Maharashtra (use the address lookup on the form). **Save**.
4. On the Order, open **Related → Order Products → Add Products**. Select **Jeans** (10#025) and **Earrings 14#004**, click **Next**, set quantity **2** for Jeans and **1** for Earrings, and **Save**.
5. Confirm on the Order:
    - **Line Count** = 2 (shown if you added it; otherwise use the formula result below)
    - **Intra State** = checked
    - **Commission Calculated** = 0, because Effective Amount is still 0
    - Totals are 0 (expected: Phase 5 populates them)
    - **Invoice Number** = INV-0001

Test B: Intra State variations

Edit the order's **Billing State** and check **Intra State** after each save:

Billing State	Expected
maharashtra	checked
MH	checked
Gujarat	unchecked
(blank)	unchecked

Set it back to Maharashtra.

Test C: Status and the Paid lock

1. On the order, change **Status** to **Confirmed**, save, then back to **Draft**. This should work.
2. Change **Status** to **Paid** and save.
    - If you get an error about the permission "Activate Orders", do the permission set fix below, then repeat.
3. On the Paid order, try **Add Products**. Expect an error.
4. Try to **Delete** the order. Expect an error.
5. Change **Status** back to **Draft**. As admin this should work.
6. If a message says you lack permission to edit an activated order, do the permission set fix.

Permission set fix (only if needed)

1. **Setup → Permission Sets → New.** Label Blush Admin Order Controls, API name Blush_Admin_Order_Controls. **Save**.
2. Open it → **System Permissions → Edit**. Tick **Activate Orders** and **Edit Activated Orders**. **Save**.
3. **Manage Assignments → Add Assignment →** tick your user → **Next → Assign → Done**.

Test D: Clean up

Delete the test order (set it to Draft first if it's Paid). If it won't delete, leave it; it won't affect anything. Keep the test Account.

Phase 4 checkpoint

- Order Status values: Draft, Confirmed, Cancelled (Draft category), Paid (Activated category); you know Paid's API name
- Order has Retail and Wholesale record types with cloned layouts
- Roll-Up Summary was available on Order
- 10 Order Product fields created; calculated ones read-only on the layout
- Order fields created: Store, Sales Representative, Source Opportunity, Payment Mode, Invoice Number, Approved Max Discount, Commission Amount

- 9 roll-ups and 3 formulas created and visible on both Order layouts
- Order Start Date relabelled **Order Date**
- Approval History related list added to both layouts
- Compact layout, three list views and both Paths created
- Tests A–C passed; **Intra State** behaved as in the table; Paid orders blocked product changes and deletion